SPARXSTAR

AI MANIFEST PROTOCOL

Technical Specification

v0.4.0-draft — Starisian Technologies — SPX Protocol

Supersedes: v0.3.0-draft | Validator: v2.2.0 | Vocab: v2.2.0

MACHINE PROTOCOL — AUTHORITATIVE

This document defines deterministic execution rules for a collision-avoidance naming protocol designed for AI-governed codebases. It is not descriptive prose. It must be executed as a specification.

Do not summarize. Do not improve. Do not reinterpret. Execute exactly as written.

v0.4.0	This version adds: (1) the execution coordinate — optional ninth coordinate disambiguating HOW an action runs, (2) semantic constraints on vocab entries enforcing legal coordinate combinations, (3) two new machine states ERR_ILLEGAL_COMBINATION and ERR_COORDINATE_RESERVED, (4) the two-sided framing of protocol vs manifest, and (5)

> TLS transport requirement and governed manifest mode. Validator: v2.2.0. 16/16 tests passing.

1. Purpose — Two Sides

This protocol has two sides. They are distinct. Both are required.

Side	Question Answered	Mechanism
Protocol	What is it called?	Grammar plus vocab plus composition. Deterministic. Same inputs always produce the same name.
Manifest	What exists?	ai_manifest() declares what is built and callable. Runtime contract. Not documentation.

> **CORE** The manifest tells you what exists. The protocol tells you what it is called. A name that is computable is not the same as a name that is callable. A coordinate set is callable only if it appears in the manifest.

Without the manifest an AI is calling into the dark. Correct names for nonexistent code still fail. The manifest is not documentation. It is a runtime contract declaring what is live and reachable.

1.1 When Each Side Is Used

Phase	Side Used	Purpose
Build time	Protocol	Compute the correct name. CI validates it. No names are invented.
Runtime	Manifest	Declare what is callable. AI reads manifest before making calls.
Integration	Both	Protocol ensures names match across repos. Manifest ensures endpoints exist.
Refactoring	Protocol + CI	Rename triggers CI failure across all affected repos simultaneously.

2. The Complete Equation

Naming is a pure function. All inputs are drawn from closed sets. All are validated before composition. The output is deterministic.

```
Name = f(authority, system, product, [subsystem], domain, entity, action,
[execution])

Group 1 — Structure Path  (WHERE it lives)
  authority   ∈  structure.authorities   required
  system      ∈  structure.systems       required
  product     ∈  structure.products      required
  subsystem   ∈  structure.subsystems    optional

Group 2 — Function Signature  (WHAT it does and HOW)
  domain      ∈  domains                required
  entity      ∈  entities               required
  action      ∈  actions                required
  execution   ∈  executions             optional

All terms must exist in system/spx-vocab.json.
No term may be invented, inferred, abbreviated, or guessed.
If any term is invalid: ERR_COORDINATE_UNDEF. Full stop.
```

2.1 The Execution Coordinate

NEW v0.4.0

execution is an optional coordinate disambiguating HOW an action runs. Without it, audio_read_stream and audio_read_batch produce the same name. With it they are different names, different classes, different routes, and therefore not a collision.

Execution	Meaning	Example
stream	Continuous real-time delivery.	artifact/audio/read/stream
batch	Deferred bulk processing.	artifact/audio/read/batch
sync	Caller waits for result.	artifact/audio/validate/sync
async	Caller continues. Result delivered later.	artifact/audio/transcribe/async
queue	Placed in ordered list.	artifact/audio/create/queue

2.2 Composition Output

```
Without execution:
  function:  spx_{auth}_{sys}_{prod}_{domain}_{entity}_{action}
  class:     SPX\{Auth}\{Sys}\{Prod}\{Dom}\{Ent}\{Act}Service
  route:     /{auth}/{sys}/{prod}/{domain}/{entity}/{action}

With execution:
```

```
    function:  spx_{auth}_{sys}_{prod}_{domain}_{entity}_{action}_{execution}
    class:     SPX\{Auth}\{Sys}\{Prod}\{Dom}\{Ent}\{Act}{Exec}Service
    route:     /{auth}/{sys}/{prod}/{domain}/{entity}/{action}/{execution}

  Examples:
    spx_brain_sparxstar_player_artifact_audio_read
    spx_brain_sparxstar_player_artifact_audio_read_stream
    spx_brain_sparxstar_player_artifact_audio_read_batch
    spx_group_aiwa_archive_lexicon_word_create
    spx_brain_sparxstar_player_streaming_artifact_audio_read_stream
```

3. ai_manifest() — The Universal Beacon

Every AI-governed repository MUST implement ai_manifest(). It is the first thing any AI calls when entering a repository. It declares what exists and what the protocol is. It is a runtime contract not documentation.

- The name is fixed. Cannot be changed namespaced or renamed per project.
- Any repository without ai_manifest() is NOT AI-governed.
- CI MUST check for its existence and fail if absent.
- MUST return complete protocol state — not a pointer to it.
- MUST be callable with zero arguments.
- All language implementations MUST return byte-for-byte compatible JSON key structure.
- legacy_map MUST be a string file path or null. Never an inline object.

3.1 Two Manifest Modes

Mode	Returns	Use Case
public	Vocabulary and grammar only. No live coordinate registry.	Open integration. Any AI can read the grammar.
governed	Vocabulary plus scoped list of live callable coordinates.	Authorized callers only. Governed by Sirus and Helios.

In governed mode the manifest returns only what the caller is authorized to see. Authorization is relational — governed by agreements between parties. No institutional gatekeeper. No approved vendor list. The system does not distinguish AI companies by identity. It distinguishes callers by what they have agreed to.

3.2 Transport — TLS Required

All calls to governed coordinates MUST be made over TLS. Certificate issuance is not gatekept by this protocol. Any caller may obtain a certificate.

```
Name   →  public. Computable from grammar. Anyone can derive it.
Call   →  private. Encrypted over TLS. Contents not readable in transit.
Access →  governed. Sirus manages agreements. Helios enforces trust.
```

3.3 Return Schema — 14 Fields, All Required

```
{
  "protocol":    string,       // Always "SPX"
  "version":     string,       // Semver e.g. "2.2.0"
  "governed":    boolean,      // Always true
  "vocab":       string,       // "system/spx-vocab.json"
  "contract":    string,       // "system/CONTRACT.md"
  "validator":   string,       // "tools/spx_validator.py"
  "domains":     object,       // Closed set with descriptions and constraints
  "entities":    object,       // Closed set with descriptions and constraints
  "actions":     object,       // Closed set with descriptions and constraints
  "executions":  object,       // Closed set with descriptions
  "synonyms":    object,       // Full map. May be {}.
  "scope":       string,       // "public" | "all"
  "mode":        string,       // "new" | "existing"
  "legacy_map":  string|null   // File path or null. Never an object.
}
```

4. Vocabulary Semantic Constraints

NEW v0.4.0

Vocab entries now carry descriptions and constraint rules. Descriptions give AI agents semantic context for coordinate selection. Constraints enforce legal coordinate combinations at the validator level. Descriptions guide selection. Constraints enforce correctness.

4.1 Why Constraints Exist

The protocol governs composition — the formula for building names. Constraints govern selection — which combinations are legally meaningful. Without constraints, transcribe on session or transcribe on wallet both produce valid-looking names. They are structurally correct but semantically illegal. Constraints catch this before CI passes bad code.

4.2 Constraint Schema

```
Entity constraints:
  "allowed_actions":  string[]  // Only these actions are valid with this
entity
  "disallowed_with":  string[]  // These actions are explicitly forbidden
  "specificity_rank": number    // Higher = more specific = preferred when
ambiguous

Action constraints:
  "allowed_domains":  string[]  // Action only valid in these domains
  "allowed_entities": string[]  // Action only valid with these entities
  "specificity_rank": number    // Higher = more specific

Domain constraints:
  "allowed_entities": string[]  // Only these entities belong in this domain
```

4.3 Constraint Enforcement

Check	Rule	Failure State
entity allowed_actions	action must be in entity's allowed_actions	ERR_ILLEGAL_COMBINATION
entity disallowed_with	action must not be in entity's disallowed_with	ERR_ILLEGAL_COMBINATION
action allowed_domains	domain must satisfy action's allowed_domains	ERR_ILLEGAL_COMBINATION
action allowed_entities	entity must satisfy action's allowed_entities	ERR_ILLEGAL_COMBINATION
domain allowed_entities	entity must satisfy domain's allowed_entities	ERR_ILLEGAL_COMBINATION

5. Machine State Taxonomy

The validator returns exactly one of seven states. Nothing else. No warnings. No suggestions. No partial passes. No repairs.

VALID	All coordinates valid. Constraints satisfied.	Execution continues.

ERR_SCHEMA_MISMATCH	ai_manifest() output missing required fields or wrong types.	CI FAILS. Repository classified NOT_AI_GOVERNED.

`ERR_COORDINATE_UNDEF`	Token not in vocabulary. No synonym mapping.	CI FAILS. Token and position reported. No fallback.
`ERR_INTENT_AMBIGUOUS`	Intent maps to more than one valid coordinate set.	CI FAILS. All matches reported. Amendment required.
`ERR_NAMESPACE_COLLISION`	Two names in same Structure Path resolve to identical coords.	CI FAILS. Both names reported. No auto-resolution.
`ERR_ILLEGAL_COMBINATION`	Coordinate pairing violates constraint rules.	CI FAILS. Violated constraint and offending pair reported.
`ERR_COORDINATE_RESERVED`	Coordinate set already registered to different logic.	CI FAILS. Existing registration reported. Amendment required.

6. Validator Test Suite — v2.2.0 Results

16 of 16 passing. All results are deterministic. Running the validator again on the same inputs produces identical output.

Test	Category	Result
SPARXSTAR player reads audio	Valid standard	PASSED
SPARXSTAR archive resolves transaction	Valid standard	PASSED
AiWA archive creates word token	Valid standard	PASSED
SPARXSTAR player reads session context	Valid standard	PASSED
SPARXSTAR editor validates audio	Valid standard	PASSED
With streaming subsystem	Valid with subsystem	PASSED
BRAIN — uppercase authority	Invalid casing	CORRECTLY FAILED
sparx-star — hyphen in system	Invalid hyphen	CORRECTLY FAILED
corporate — invented authority	Invalid not in vocab	CORRECTLY FAILED
authenticate — unknown action	Invalid not in vocab	CORRECTLY FAILED

Test	Category	Result
transcribe on session	Invalid illegal combo	CORRECTLY FAILED
transcribe on word	Invalid illegal combo	CORRECTLY FAILED
audio/read/stream — with execution	Valid with execution	PASSED
audio/read/batch — with execution	Valid with execution	PASSED
real-time — hyphen in execution	Invalid hyphen	CORRECTLY FAILED
turbo — invented execution	Invalid not in vocab	CORRECTLY FAILED

7. Open Questions — Deferred

Question	Deferred Until
Preflight reservation check before minting new coordinates	v0.5.0
Intent-to-coordinate normalization — governed pre-filter layer	v0.5.0
Binary coordinate buffer + SHA-256 for cross-vendor verification	v1.0.0
Structure path: auto-derived from repo root or explicit in vocab	Before install script
Neo4j and PostgreSQL composition rules	Before database enforcement
Governance model for universal dictionary amendments	Before public release
Cryptographic authority signing — governed manifest JWT	v1.0.0

Final Rules

> ***The manifest tells you what exists. The protocol tells you what it is called.***
>
> ***Same input + same vocab = SAME output. If not, the protocol is incomplete.***

> ***A validator that mutates state is not a validator. It is an unreliable executor.***

Validator: spx_validator.py v2.2.0 | Vocab: spx-vocab.json v2.2.0 | Spec: v0.4.0-draft

Generated: April 2026 | Claude Sonnet 4.6 | Starisian Technologies